Employee Salary Calculator

Project Report • Python + Flask • Internship Submission

Project objective: Create a program that calculates employee gross salary, deductions, and net salary using configurable salary rules.

Learning objective: Practice dictionaries, functions, conditions, validation, and business-rule implementation while keeping the rules separate from input handling.

Item	Implementation
Language	Python 3
Web framework	Flask
Production server	Gunicorn
Testing	pytest
Deployment	Render Web Service
Source control	Git + GitHub
Health check	/health

1. Requirements and Deliverables

The project addresses the assignment requirements for a salary calculator, sample employee records, and a salary calculation breakdown. Salary rules are kept in a dedicated configuration dictionary so changes can be made without rewriting the calculation flow.

Deliverables included: Flask web calculator; sample employee records in JSON; salary breakdown; automated tests; README and deployment runbook; Render deployment configuration; Git ignore rules; and this project report PDF.

2. Application Architecture

app.py handles HTTP routes, form input, validation errors, result rendering, and the deployment health endpoint. **salary_calculator.py** contains reusable business logic. **salary_rules.py** contains configurable allowances, deductions, and tax brackets. HTML templates and CSS provide the user interface.

This separation keeps business rules independent from input handling, making the calculation functions reusable and independently testable.

Project structure:

```
app.py
salary_calculator.py
salary_rules.py
requirements.txt
```

```
render.yaml
templates/index.html
templates/result.html
static/style.css
data/sample_employees.json
tests/test_salary_calculator.py
tests/test_app.py
docs/DEPLOYMENT_RUNBOOK.md
```

3. Configurable Salary Rules

Category	Rule	Rate / Condition
Allowance	Housing	20% of base salary
Allowance	Transport	10% of base salary
Allowance	Performance Bonus	5% of base salary
Deduction	Provident Fund	12% of gross salary
Deduction	Insurance	2% of gross salary
Tax	Bracket 1	10% up to 50,000
Tax	Bracket 2	15% from 50,000 to 100,000
Tax	Bracket 3	20% above 100,000

Important: these percentages and tax brackets are sample educational rules created for this project. They are not statutory payroll rules or financial/legal advice.

4. Calculation Method

1. Read and validate the employee record.
2. Calculate each allowance from base salary.
3. Gross salary = base salary + total allowances.
4. Calculate percentage deductions from gross salary.
5. Calculate progressive income tax using the configured brackets.
6. Total deductions = percentage deductions + income tax.
7. Net salary = gross salary – total deductions.

5. Sample Calculation

For a sample employee with a base salary of INR 60,000:

Component	Calculation	Amount (INR)
Base salary	Given	60,000.00
Housing	20% × 60,000	12,000.00
Transport	10% × 60,000	6,000.00
Performance bonus	5% × 60,000	3,000.00
Gross salary	60,000 + 21,000	81,000.00
Provident Fund	12% × 81,000	9,720.00
Insurance	2% × 81,000	1,620.00

Component	Calculation	Amount (INR)
Income tax	10% of first 50,000 + 15% of next 31,000	9,650.00
Total deductions	9,720 + 1,620 + 9,650	20,990.00
Net salary	81,000 – 20,990	60,010.00

6. Validation and Testing

Input validation checks required employee ID, name, department, and a positive numeric base salary. Tests cover invalid salary input, progressive tax brackets, complete salary calculations, and the Flask health endpoint/application response.

The deterministic core salary calculations were checked during package preparation for the configured brackets and the INR 60,000 example. Full Flask/pytest execution should be run locally after installing the project's requirements.

7. Deployment Configuration

The repository includes **render.yaml** so the expected Render configuration is stored with the source code. The production start command is **gunicorn app:app**, and the service health check is **/health**. The README and deployment runbook contain the Windows PowerShell commands for local setup, GitHub push, Render deployment, verification, and rollback evidence.

8. GitHub and Submission Plan

Recommended repository name: **employee-salary-calculator**. Recommended GitHub description: *A Python Flask employee salary calculator with configurable allowances, deductions, progressive tax rules, validation, tests, and Render deployment configuration.*

Submit the public GitHub URL, live Render URL, project ZIP/report if requested, and the evidence screenshots required by the internship portal.

9. Rollback Evidence

Rollback evidence must come from the real Render deployment history and should not be fabricated. After the first successful deployment, keep it as the known-good version. If a later deployment is made, use Render's Deploys page to roll back to the earlier successful deploy when required, verify the live app, and capture the deploy history/rollback result plus the relevant commit SHAs.

10. Interview Questions and Answers

How would you represent employee data? Use dictionaries for individual employee records and a list of dictionaries for multiple employees; this is readable and easy to serialize as JSON.

Why should business rules be separated from input handling? It keeps the request/UI layer focused on validation and presentation while the calculation layer remains reusable and testable.

How would you modify the program if salary rules changed? Update salary_rules.py and the expected test values; the ordinary calculation flow does not need to be rewritten.

11. Limitations and Future Enhancements

This is an educational single-employee workflow. Future versions could add database storage, bulk CSV upload, authentication, audit logs, downloadable payslips, versioned rules, and a dedicated API. Real payroll use would require verified jurisdiction-specific rules and professional review.

12. Final Submission Checklist

- ■ Public GitHub repository
- ■ README.md present
- ■ render.yaml committed
- ■ Local tests pass
- ■ Live Render URL works
- ■ Salary result screenshot
- ■ Health endpoint screenshot
- ■ Deployment/rollback evidence if required
- ■ ZIP and PDF attached to the internship portal if requested

Prepared for internship project submission • 05 October 2026